

Web Scraping — Internship Assignment

Role: Scraper / Data Collection Intern · Duration: 5–7 Days · Stack: Python · requests / Selenium · BeautifulSoup · JSON

1. Normalised Job Listing Schema

Every listing you collect — regardless of company or platform — must be stored in this format. All fields are required.

Field	Type	Description
title	str	Job title as listed on the career page
company	str	Company name
location	str	City / region (e.g. "Bangalore", "Pan India")
work_mode	str	"remote" "hybrid" "on-site"
seniority	str	"intern" "junior" "mid" "senior"
business_unit	str	Division / team (e.g. "Engineering", "Data Science")
required_skills	list[str]	Skills extracted from the JD (e.g. ["Python", "SQL"])
preferred_skills	list[str]	Good-to-have skills mentioned in the JD
description	str	Full job description text
apply_url	str	Direct link to the application page
scraped_at	str	ISO 8601 timestamp with UTC offset (e.g. 2025-06-12T09:45:32+05:30)

2. Platform Types in Scope

The 28 target companies span 8 different platform architectures. Your scraper must handle whichever platform type the company you choose uses.

Platform Type	How it works	Companies in scope
Workday ATS	POST JSON API; offset/limit pagination; no JS rendering needed	Accenture, Wells Fargo, Mastercard, Novartis, Eli Lilly, Salesforce, Capgemini, Fidelity, HCL, RTX
SmartRecruiters ATS	REST GET API; offset pagination; returns JSON arrays	Syngenta, Continental, ServiceNow, Sanofi
Greenhouse API	Cursor-based pagination; GET /v1/boards/{company}/jobs	Stripe
Eightfold AI API	Undocumented REST API; reverse-engineer from browser XHR traffic	Morgan Stanley, American Express
Custom JSON / Azure API	Proprietary back-end APIs; schema varies per company	Microsoft, IBM

Platform Type	How it works	Companies in scope
Selenium — Standard	JS-rendered career pages; WebDriverWait; handle infinite scroll	Apple, Google, Wipro, Synopsys
Selenium — Complex / Wall	Anti-bot measures or login walls; stealth driver techniques required	Goldman Sachs, Atlassian, TCS, Infosys BPM
XML Feed	Structured XML sitemap; parse with lxml; handle XML namespaces	Cognizant

3. Companies — Ordered by Difficulty (Hardest First)

All 28 companies are in scope. **Your score depends on which ones you choose to scrape** — harder targets earn more points. Read Section 4 before deciding.

Tier key: **Expert — 40 pts** **Hard — 30 pts** **Medium — 20 pts** **Starter — 10 pts**

#	Tier	Company	Platform	Target URL / Notes	Script Filename
1	Expert	Goldman Sachs	Selenium (custom)	higher.globalsign.com Aggressive bot-detection; stealth headers & randomised delays required	goldman_scraper.py
2	Expert	Atlassian	Selenium (custom)	atlassian.com/company/careers Heavy JS SPA; dynamic pagination; strict rate-limiting	atlassian_scraper.py
3	Expert	Morgan Stanley	Eightfold AI API	morganstanley.eightfold.ai Undocumented API; must reverse-engineer network traffic; returns 1 row currently	morganstanley_scraper.py
4	Expert	American Express	Eightfold AI API	aexp.eightfold.ai Eightfold endpoint — no public docs; inspect XHR; no output yet	amex_scraper.py
5	Expert	Stripe	Greenhouse API	boards-api.greenhouse.io/stripe Non-standard cursor pagination; no output yet	stripe_scraper.py
6	Expert	Microsoft	Custom JSON API (Azure-backed)	gcsservices.careers.microsoft.com Proprietary API; complex filter & pagination schema; no output yet	microsoft_scraper.py
7	Expert	TCS	Selenium + login wall	ibegin.tcs.com / tcs.com/careers Registration wall; session handling required; HTML artifacts in current output	tcs_scraper.py
8	Hard	Synopsys	Selenium (Avature)	synopsys.avature.net Avature portal; AJAX-rendered cards; only 6 jobs returned so far	synopsys_scraper.py
9	Hard	Google	Selenium (custom SPA)	google.com/about/careers JS SPA; currently ~20 jobs — pagination incomplete	google_scraper.py
10	Hard	IBM	Custom JSON API	jobsapi-google.m-cloud.io Third-party hosted API; schema unclear; currently 1 row	ibm_scraper.py
11	Hard	Apple	Selenium (custom)	jobs.apple.com/en-in/search JS-heavy; infinite scroll; 374 jobs when working correctly	apple_scraper.py

#	Tier	Company	Platform	Target URL / Notes	Script Filename
12	Hard	Infosys BPM	Selenium (partially walled)	career.infosys.com 190 jobs collected; some pages prompt login — handle gracefully	infosys_scraper.py
13	Hard	Wipro	Selenium (custom)	careers.wipro.com 26 jobs collected; more pages likely available	wipro_scraper.py
14	Hard	RTX	Workday API (non-standard)	globalhr.wd5.myworkdayjobs.com Workday variant; no output yet — endpoint needs verification	rtx_scraper.py
15	Medium	Cognizant	XML Feed	careers.cognizant.com (XML feed) 380 jobs; parse namespaced XML with lxml	cognizant_scraper.py
16	Medium	HCL Technologies	Workday API	hcltech.wd3.myworkdayjobs.com Standard Workday; currently 1 row — offset logic needs fix	hcl_scraper.py
17	Medium	Syngenta	SmartRecruiters API	api.smartrecruiters.com (SyngentaGroup) 116 jobs; company identifier must be exact	syngenta_scraper.py
18	Medium	Continental	SmartRecruiters API	api.smartrecruiters.com (continental) 300 jobs; standard offset + limit pagination	continental_scraper.py
19	Medium	ServiceNow	SmartRecruiters API	careers.smartrecruiters.com/servicenow 141 jobs; filter by India location	servicenow_scraper.py
20	Medium	Sanofi	SmartRecruiters API	sanofi.wd3.myworkdayjobs.com 110 jobs; standard SmartRecruiters with India filter	sanofi_scraper.py
21	Starter	Accenture	Workday API	accenture.wd103.myworkdayjobs.com 12,584 jobs; standard Workday POST; offset pagination	accenture_scraper.py
22	Starter	Wells Fargo	Workday API	wf.wd1.myworkdayjobs.com 203 jobs; clean Workday API	wellsfargo_scraper.py
23	Starter	Mastercard	Workday API	mastercard.wd1.myworkdayjobs.com 118 jobs; clean Workday API	mastercard_scraper.py
24	Starter	Novartis	Workday API	novartis.wd3.myworkdayjobs.com 117 jobs; clean Workday API	novartis_scraper.py
25	Starter	Eli Lilly	Workday API	lilly.wd5.myworkdayjobs.com 46 jobs; clean Workday API	elililly_scraper.py
26	Starter	Salesforce	Workday API	salesforce.wd12.myworkdayjobs.com 37 jobs; clean Workday API	salesforce_scraper.py
27	Starter	Capgemini	Workday API	capgemini.wd3.myworkdayjobs.com 25 jobs; standard Workday; filter for India	capgemini_scraper.py
28	Starter	Fidelity Investments	Workday API	fmr.wd1.myworkdayjobs.com 30 jobs; clean Workday API	fidelity_scraper.py

4. Scoring — Company Selection

40% of your total mark comes from **which companies you chose to scrape**. Harder targets earn more points per scraper. Easier tiers are capped so that staying safe cannot inflate your score.

4a. Points per Tier

Tier	Points	What makes it hard	Score cap	Counts toward
Expert	40 pts	Bot detection, login walls, broken/undocumented APIs, reverse-engineering required	No cap	Full score
Hard	30 pts	Proprietary APIs, SPA Selenium, partial data, session / auth handling	No cap	Full score
Medium	20 pts	SmartRecruiters pagination, XML feeds, non-standard Workday subdomains	Max 3	Up to 3 companies
Starter	10 pts	Clean Workday JSON API with documented offset pagination	Max 2	Up to 2 companies

4b. Calculation Rules

- Sum points for each company you scrape with at least 20 clean, schema-valid listings.
- **Starter tier:** maximum 2 companies count toward your score, even if you scrape more.
- **Medium tier:** maximum 3 companies count toward your score.
- **Hard and Expert:** no cap — every successful scraper scores.
- A scraper returning fewer than 20 valid listings, or with schema violations, scores 0 for that company.

4c. Score Examples

Scenario	Score	Tiers used	Interpretation
Goldman Sachs + Microsoft + Atlassian	120 pts	Expert × 3	Maximum difficulty — every company counts
Goldman Sachs + Apple + Accenture + Wells Fargo	100 pts	E + H + S×2 (capped)	Hard anchor; 2 Starters count, any extra ignored
Google + IBM + Sanofi + Cognizant	100 pts	H×2 + M×2	Solid breadth across platform types
Apple + Infosys + Wipro	90 pts	Hard × 3	All hard platforms — no easy shortcuts
Accenture + Novartis + Mastercard + Capgemini	20 pts	Starter × 4 (capped at 2)	Only 2 Starters count — 2 companies wasted

Tip: a single working Expert scraper (40 pts) outscores four Starter scrapers (capped at 20 pts). Prioritise harder targets.

5. Your Tasks

Task 1 — Choose Your Companies

Select any combination from the table in Section 3. You must scrape at least one API-based and at least one Selenium-based company. There is no upper limit on how many you attempt.

Task 2 — Build and Run the Scrapers

For each company you choose, write a standalone Python script that:

- Discovers and calls the underlying API, or renders the page via Selenium.
- Paginates through all available results.
- Collects at least 20 live listings.
- Outputs data in the normalised schema defined in Section 1.

Task 3 — Normalise the Data

Map every field the source returns into the standard schema. Parse `required_skills` and `preferred_skills` as Python lists — never a string. Add a `scraped_at` ISO 8601 timestamp to every record.

Task 4 — Save Output

Write all results to `jobs_output.json` (a single JSON array). Save a `run_log.txt` that logs pages fetched, job counts per company, and any errors encountered.

6. Aligning Your Output with Ours

We have existing scrapers for every company in the table. Your output must drop into our pipeline without manual cleaning. Follow these rules exactly:

- **Script filename:** use the exact name from the Script Filename column.
- **Field names are case-sensitive:** `required_skills` not `skills`, not `requiredSkills`.
- Skills must always be Python lists, never a comma-separated string.
- `scraped_at` must be ISO 8601 with UTC offset, e.g. `2025-06-12T09:45:32+05:30`.
- `apply_url` must be the direct job application URL, not the search results page.

7. Expected Output

`jobs_output.json` — example entry

Every entry must follow this exact structure:

```
[
  {
    "title": "Associate Software Engineer",
    "company": "Accenture",
    "location": "Bangalore",
    "work_mode": "hybrid",
    "seniority": "junior",
    "business_unit": "Technology",
    "required_skills": ["Java", "Spring Boot", "REST APIs", "SQL", "Git"],
    "preferred_skills": ["Microservices", "Docker", "Agile"],
    "description": "Full job description text here...",
    "apply_url": "https://accenture.wd103.myworkdayjobs.com/en-US/...",
    "scraped_at": "2025-06-12T09:45:32+05:30"
  }
]
```

`run_log.txt` — example

```

[2025-06-12 09:45:00] Starting Accenture scraper (Workday API)...
[2025-06-12 09:45:08] Page 1: 20 jobs fetched
[2025-06-12 09:45:11] Page 2: 20 jobs fetched
[2025-06-12 09:45:13] Page 3: 14 jobs fetched — end of results
[2025-06-12 09:45:13] Accenture: 54 jobs collected
[2025-06-12 09:46:00] Starting Goldman Sachs scraper (Selenium)...
[2025-06-12 09:47:15] Page 1 loaded — 20 job cards found
[2025-06-12 09:48:02] Page 2 loaded — 20 job cards found
[2025-06-12 09:48:50] Page 3 loaded — 12 job cards found — no next page
[2025-06-12 09:48:51] Goldman Sachs: 52 jobs collected
[2025-06-12 09:48:52] Total jobs saved to jobs_output.json: 106
[2025-06-12 09:48:52] Done.

```

8. Evaluation Criteria

Criterion	What we look for	Weight
Company Selection Score	Points earned from difficulty tier of chosen companies (see Section 4)	40%
Schema Compliance	All required fields present and correctly typed across every listing	20%
Data Quality	Skills parsed as lists; full-text descriptions, not snippets	15%
Error Handling	Network errors, missing fields, pagination edge cases handled gracefully	15%
Code Cleanliness	Readable functions, sensible variable names, inline comments	10%

9. Hints & Tips

- **Finding the API:** F12 → Network → Fetch/XHR → scroll the careers page → identify the JSON request → copy that URL.
- **Workday pattern:** POST to `https://company.wdX.myworkdayjobs.com/wday/cxs/company/CareerPage/jobs` with body `{"limit": 20, "offset": 0}`.
- **SmartRecruiters pattern:** GET `https://api.smartrecruiters.com/v1/companies/{identifier}/postings?limit=100&offset=0`.
- **Greenhouse pattern:** GET `https://boards-api.greenhouse.io/v1/boards/{company}/jobs?content=true` for full descriptions.
- **Eightfold pattern:** Inspect XHR for `/api/apply/v2/jobs` or similar — no public docs, parameters vary.
- **Selenium stealth:** use undetected-chromedriver or execute `navigator.webdriver = undefined` via JS for bot-protected sites.
- **Always WebDriverWait:** never `time.sleep()`. Use `expected_conditions.presence_of_element_located()`.
- **Rate limiting:** add 0.5–1 s between requests. Never send rapid-fire calls to the same server.
- **Skill extraction:** use keyword matching against a known list (Python, SQL, Java, AWS...) when the JD has no dedicated skills field.

Important: Only scrape publicly accessible job listings — no login required, no paywalled content.

10. What to Submit

1. Python scraper files — one per company, named exactly as shown in the Script Filename column.
2. jobs_output.json — actual scraped output (minimum 20 valid entries per company).
3. run_log.txt from your most recent run.
4. README.md explaining how to install dependencies and run each scraper.
5. Push everything to a public GitHub repo and share the link.

Questions? Reply to your Internshala message thread.

We review submissions on a rolling basis — quality over speed.